סיכום Skills

מוסכמות הפיתוח בפרויקט ניהול אנשי קשר

`expertise-code-quality` `expertise-nodejs-typescript`

`expertise-api-rest` `expertise-postgres-drizzle`

`expertise-observability` `expertise-react-nextjs`

`expertise-testing`

נכתב עבור ראש הצוות · 22.10.2026

מסמך זה מסכם שבעה Skills של Claude Code המגדירים את מוסכמות הפיתוח בפרויקט ניהול אנשי הקשר (`Next.js` / `NestJS` / `Drizzle` / `PostgreSQL` / `Zod`). כל סקיל מכסה שכבה אחרת, ויחד הם קובעים את סטנדרט הקוד המחייב בפרויקט. סקיל אחד, `expertise-code-quality`, פועל על כל קוד, וכל סקיל אחר נטען כשנוגעים בתחום שלו. המטרה: תמונת מצב מהירה וברורה, ללא צורך לקרוא את קובצי הסקילים המלאים.

1 עקרונות איכות ומבנה הפרויקט

`expertise-code-quality`

> **היקף:** חל על **כל קובץ** בפרויקט, בכל שכבה. הוא הבסיס שעליו בנויים שאר הסקילים, וכולל את מפת התיקיות של הפרויקט.

- **פשטות על פני תחכום:** לממש את הפתרון הישיר לדרישה של היום, בלי הנדסת-יתר לצורך שעדיין לא קיים.
- **אין over-engineering:** זה MVP לארגון אחד. מערכת `plugin` או `config` גנרית לדבר שמשמש פעם אחת היא באג ולא צניעות.
- **אחריות יחידה ושמות בעלי משמעות:** לא `data` / `temp` / `handleStuff` לכל דבר שאינו טריוויאלי.
- **הערות מסבירות "למה" ולא "מה":** קוד טוב כבר מסביר מה הוא עושה.
- **שגיאות קולניות בפיתוח, בלי דליפת פנימיות בפרודקשן:** אסור לבלוע exceptions, ואסור לחשוף `stack trace` או `SQL` למשתמש.
- **כל פעולה ב-backend כוללת לוג:** שינוי מצב, סירוב מכוון וכל נתיב שגיאה, עם מזהים בלבד ובלי מידע אישי.
- **מבנה ארבע שכבות:** `apps/web` להצגה בלבד, `apps/api` לכל ה-API ושכבת השירות, ורק ה-`service` ניגש ל-`packages/db`. אין שכבת Repository נפרדת.
- **משותף מול פיצ'ר:** קוד שמשמש פיצ'ר אחד יושב בתיקייה שלו. ברגע שפיצ'ר שני צריך אותו, הוא עובר למקום המשותף (`packages/ui`, `packages/shared-schemas`).

> **צ'ק-ליסט לפני מסירה:** האם עמית מהצוות יבין את הקוד בקריאה אחת? האם יש גרסה פשוטה יותר שעונה על הדרישה? האם הקוד ישב בשכבה ובתיקייה שהמפה מגדירה?

2 Zod-ו Node.js, TypeScript

`expertise-nodejs-typescript`

> **היקף:** קוד backend ב-`.ts` / `.js`, הגדרות `tsconfig`, `npm workspaces` ו-`turbo.json`, וסכמות Zod והטיפוסים שנגזרים מהן.

- **TypeScript במצב strict מלא:** `strict`, `noUncheckedIndexedAccess`, `exactOptionalPropertyTypes` ועוד. `tsc --noEmit` הוא שער CI נפרד מ-Jest.
- **אין `any` ואין `@ts-nocheck`:** בגבולות אמון (JSON גולמי, גוף בקשה) משתמשים ב-`unknown` ומצמצמים. אין `enum` ו-`namespace`, אלא אובייקט `as const` ו-union נגזר.
- **הסכמה היא הטיפוס:** Zod 4 משמש גם ב-`apps/web` (React Hook Form) וגם ב-`apps/api` (Pipe). הסכמות יושבות ב-`packages/shared-schemas/src/<resource>/`, וב-`types.ts` יש רק כינויי `z.output` / `z.input`, בלי `interface` ידני מקביל.
- **כל הודעה שמשתמש רואה היא בעברית** וחיה בסכמה, לא ברכיב. בדיקה אוטומטית (`hebrew-messages.test.ts`) נכשלת אם הודעה חסרה אותיות עבריות.
- **`null` מול `undefined`:** ב-`.set()` של Drizzle מפתח חסר פירושו "ללא שינוי" ו-`null` פירושו "הגדר NULL". אלה שלושה מצבים נבדלים שחובה להחליט עליהם בכל גבול.
- **מודולים:** ESM בכל מקום, חוץ מ-`apps/api`, `packages/db` ו-`packages/shared-schemas` שהם CommonJS, כי ההזרקה של NestJS מסתמכת על כך.
- **`npm workspaces`:** כל חבילה מצהירה על כל תלות ישירה משלה, גם אם היא נפתרת דרך hoisting. מתקינים מהשורש עם `--workspace`.
- **שגיאות ואבטחה:** לא בולעים שגיאה בשקט (זורקים מחדש עם `Error.cause`), לוגים רק דרך `Logger` ולא `console.log`, סודות רק בקובץ `env` מוחרג מגיט, ו-`npm audit` ב-CI.

3 REST API (NestJS) עיצוב

`expertise-api-rest`

> **היקף:** כל מה שתחת `apps/api/src/**`: מודולים, controllers, services, Pipes, Filters, Swagger ו-health. לא כולל קוד `apps/web` ולא שאילתות Drizzle.

- **שלישיית הקבצים לכל משאב:** `x.module.ts` + `x.controller.ts` + `x.service.ts`, ומודול שלא יובא ב-`app.module.ts` אינו קיים עבור האפליקציה.
- **דק Controller:** בקשה, ולידציה (Pipe), קריאה אחת ל-service והחזרה. כללים עסקיים וגישה לנתונים יושבים ב-service.

- **`Contact` לא נמחק לעולם:** אין `DELETE /contacts/:id`. הפעולות הן `POST /contacts/:id/deactivate` ו-`/activate`, והן מחזירות את איש הקשר המעודכן.
- **כלל הכרעה לפעולות לא-CRUD:** אם לפעולה יש מחזור חיים, תופעות לוואי או סיבת דחייה משלה, היא `POST` על תת-נתיב בשם פועל. אם זה רק שינוי שדה או קשר, זה `PATCH` / `PUT` / `DELETE` רגיל.
- **ולידציה בנקודה אחת:** כל גוף ו-query עוברים ב-`ZodValidationPipe` המשותף, עם אותה סכמה שה-Frontend משתמש בה.
- **מעטפת תגובה אחת:** `{ data }` להצלחה ו-`{ error: { code, message, details } }` לשגיאה, דרך `ResponseInterceptor` ו-`AllExceptionsFilter`. `error.message` תמיד בעברית.
- **קודי סטטוס:** `400` ולידציה, `404` לא נמצא (גם לרשומה מחוץ לדומיין, כדי לא לחשוף קיום), `409` התנגשות, `429` הגבלת קצב, `503` לא מוכן.
- **Swagger נוצר מסכמות Zod:** דרך `@ApiZodBody` ו-`@ApiZodResponse` ודומיהם, בלי מחלקות DTO ידניות. נגיש ב-`/docs`.
- **Health:** `/health/live` לא נוגע בתלויות, ו-`/health/ready` בודק את מסד הנתונים ומחזיר `503` אם הוא לא זמין.
- **גרסאות והגבלת קצב:** אין `/v1` כרגע. הנתיב הציבורי `POST /registration-requests` יחייב הגבלת קצב (טרם נבנה).

> **תרשים מחלקות:** כשמוסיפים, מסירים או משנים שם של מודול, controller, service, פעולה ציבורית או תלות, מרעננים את `docs/diagrams/class-diagram.md`. התרשים נוצר מהקוד ולא נערך ידנית.

4 DB: Drizzle-ו PostgreSQL-ו סכמה

`expertise-postgres-drizzle`

> **היקף:** הסכמה והמיגרציות ב-`packages/db`, ושאילתות Drizzle בתוך services של `apps/api`.

כללים מחייבים

- **`Contact` = soft-delete בלבד:** אסור `db.delete(contacts)`. "מחיקה" היא `status = INACTIVE`, ורשימות מסוננות ל-`ACTIVE` כברירת מחדל. יש בדיקה שמוודאת שלא נשלח `DELETE` על הטבלה.
- **handle אחד ל-DB:** `client.ts` יוצר את ה-`Pool` וה-`drizzle()` היחידים. אופרטורים ושאילתות מיובאים מ-`@contact-management/db` ולא מ-`drizzle-orm`.
- **כתיבה מרובת טבלאות בתוך `db.transaction`:** כל פקודה בתוכה משתמשת ב-`tx`, וכשל גורר חזרה לאחור.
- **תרגום שגיאות driver ב-service:** הפרת `unique` (`23505`) הופכת ל-`ConflictException`, ותוצאה ריקה הופכת ל-`NotFoundException`. שגיאה גולמית לא מגיעה ללקוח.

- **כ-`sql` tagged template בלבד:** אף פעם לא `sql.raw()` או שרשור מחרוזות על קלט שמקורו במשתמש.

עיצוב סכמה ושאילתות

- **כללי עסק ב-DB:** `SupportedBody.companyId` (ח"פ) הוא `UNIQUE` ברמת מסד הנתונים, ולא רק בקוד. `Domain` היא טבלה ולא enum, כדי שאפשר יהיה לשנות תחומים בלי מיגרציה.
- **ללא N+1:** טוענים את הילדים של כל העמוד בשאילתה אחת עם `inArray`, ושולפים רק את העמודות שהמסך צריך.
- **סינון לפי דומיין:** יהיה דרך helper ייעודי לכל מודל (`scopeContact`, `scopeSupportedBody`), כי לכל מודל מסלול שונה לדומיין. עדיין לא נבנה.
- **Audit trail אינו פעיל כרגע:** המנגנון הקודם (Prisma `$extends`) הוסר, ושום דבר לא כותב ל-`AuditLog`. אסור לכתוב "מתועד אוטומטית".
- **מיגרציות:** `drizzle-kit generate` ו-`migrate`. קובצי ה-SQL ו-`meta/` נכנסים ל-git עם שינוי הסכמה, ולא עורכים מיגרציה שכבר הורצה.

> **תרשים ERD:** כל שינוי ב-`packages/db/src/schema/index.ts` כולל רענון של `docs/diagrams/erd.md`. התרשים נוצר מהסכמה ולא נערך ידנית.

5 לוגים ו-OpenTelemetry

`expertise-observability`

> **היקף:** כל פעולה שמוסיפים או משנים ב-`apps/api`: endpoint, פעולת service, נתיב שגיאה, job, startup ו-shutdown. מחייב לפי `AGENTS.md`.

- **כלל ברזל: אף פעם לא מידע אישי בלוג.** לא שמות, ת"ז, טלפונים, מיילים, הערות, מונחי חיפוש או גוף בקשה. רושמים **מזהים וספירות בלבד**. ה-`LoggingInterceptor` רושם רק את תבנית הנתיב (`/contacts/:id`).
- **לוגים דרך `Logger` של Nest:** מופע אחד לכל מחלקה, ואף פעם לא `console.log`.
- **כל בקשה מקבלת שורת כניסה ושורת יציאה** מה-`LoggingInterceptor` הגלובלי, וכותרת `x-trace-id` שמאפשרת לקשר דיווח של משתמש ללוגים.
- **כל נתיב שגיאה נרשם פעם אחת:** תקלה לא צפויה ב-`error` עם stack, שגיאת לקוח (4xx) ב-`warn` עם סטטוס וקוד בלבד, וסירוב מכוון של service ב-`warn` במקום שבו ההחלטה התקבלה.
- **רמות:** שינוי מצב מוצלח ב-`log`, סירוב או התנגשות ב-`warn`, כשל בתלות ב-`error`. קריאות בלבד לא דורשות שורה נוספת.

- **ה-import הראשון:** `./telemetry/instrumentation` חייב להיות ה-import הראשון ב-`main.ts` אחרי `dotenv/config`. אחרת ה-tracing מושבת בשקט.
- **תצורה:** `LOG_LEVEL`, `LOG_FORMAT` ו-`OTEL_EXPORTER_OTLP_ENDPOINT`. בלי כתובת collector שום דבר לא יוצא מהתהליך, ועדיין יש trace id מקומי.
- **כיבוי מסודר:** סגירת ה-pool ורוקן נתוני ה-telemetry ב-SIGTERM.

> **שינוי לא נחשב גמור:** שינוי שמוסיף פעולה בלי לוג מתאים, או שרושם מידע אישי, אינו גמור, בדיוק כמו שינוי בלי בדיקות.

6 Next.js (apps/web)-ו React

`expertise-react-nextjs`

> **היקף:** רכיבים, דפים, layouts ו-Redux ב-`apps/web`. האפליקציה היא **הצגה בלבד**: אין Route Handlers ואין גישה ל-`packages/db`.

- **כל הגישה לנתונים דרך `apps/api`:** רק דרך `lib/api-client.ts`, בלי `fetch` עם כתובת קשיחה בתוך רכיב.
- **Server Component כברירת מחדל:** `"use client"` רק כשיש צורך אמיתי ב-hooks, באירועים או ב-API של הדפדפן, ורק בעלה הקטן ביותר. שליפת נתונים לא נעשית ב-`useEffect`.
- **מוטציות דרך ה-API, לא Server Actions:** כל שינוי נתונים הוא endpoint ב-`apps/api` שנקרא ב-RTK Query. רענון מטמון נעשה ב-`invalidatesTags` של RTK Query, לא ב-`revalidatePath`.
- **Mantine + RTL:** האפליקציה בעברית בלבד. `DirectionProvider` + `MantineProvider` ו-`dir="rtl"` על `<html>`, פעם אחת בלבד ב-root layout.
- **Redux Toolkit:** ה-`<Provider>` יושב בתוך Client Component. נתוני DB לא נכנסים ל-slice רגיל, כי RTK Query מנהל אותם.
- **React Hook Form + Zod:** הטופס משתמש בסכמת **הבקשה** (לא סכמת התגובה). רכיבי Mantine שמחזירים ערך גולמי (`Select`, `MultiSelect`, `DatePickerInput`) עוטפים ב-`Controller`.
- **כל מה שהמשתמש קורא הוא בעברית:** תוויות, הודעות שגיאה ודפי `error.tsx` ו-`not-found.tsx`. לא מציגים `Error.message` גולמי.
- **איכות:** נגישות היא דרישה ראשונה (תוויות, אלמנטים סמנטיים, מקלדת), בלי god components, ובלי `useMemo` / `useCallback` לפני שיש בעיה מוכחת.

7 Jest ובדיקות

`expertise-testing`

> **היקף:** כל קובץ בדיקה והגדרת Jest, בשני היישומים. מחייב לפי `AGENTS.md` לפני יצירה, עריכה או סקירה של בדיקה.

- **TDD הוא השיטה המחייבת:** כותבים קודם בדיקה שנכשלת, מוודאים שהיא נכשלת מהסיבה הנכונה, ורק אז ממשים. לא כותבים כמה בדיקות ואז כמה מימושים.
- **Jest + React Testing Library בלבד:** בלי Vitest או מריץ אחר. ב-`apps/web` משתמשים ב-`next/jest`, וב-`apps/api` ב-`@nestjs/testing`.
- **בדיקות service ב-`apps/api`:** עם `test-utils/db-mock.ts`, שמייצר SQL אמיתי של Drizzle ומדמה רק את הביצוע. בודקים את ה-SQL והפרמטרים, בלי מסד נתונים.
- **רינדור רק דרך העטיפה המשותפת:** `test-utils/render.tsx` (עם `MantineProvider`), לא `render` גולמי מ-RTL.
- **פער מרכזי:** Jest לא יכול לרנדר Server Components אסינכרוניים. מחלצים את הלוגיקה לפונקציה רגילה ובודקים אותה, ורינדור מלא נבדק ב-E2E.
- **מיקום קבצים:** `*.test.ts(x)` צמוד לקובץ המקורי. תיקיות ייעודיות רק ל-`test-utils/`, `__mocks__/` ובדיקות `integration/`.
- **סדר עדיפויות לפי מחיר הטעות:** הרשאות וסינון דומיין, **פרטיות בלוגים ובשגיאות**, soft-delete, ולידציות Zod של הטפסים, ומעברי מצב של בקשות רישום. לא רודפים אחרי אחוז כיסוי כללי.
- **נורמות:** שם הבדיקה מסביר את הכשל בלי לפתוח את הקוד, הבדיקות עצמאיות, ולא משאירים `.only` או `.skip`.

טבלת סיכום מהירה

Skill	מה הוא מגדיר	נטען כש...
`expertise-code-quality`	עקרונות איכות ומבנה הפרויקט	תמיד, בכל קוד
`expertise-nodejs-typescript`	טיפוסים ← strict, Zod, ESM, null/undefined, npm workspaces	קוד backend, סכמות, הגדרות
`expertise-api-rest`	מודולים, URL, מעטפת תגובה, Swagger, health, תרשים מחלקות	כל מה שב-apps/api
`expertise-postgres-drizzle`	soft-delete, transactions, מיגרציות, סינון דומיין, תרשים ERD	סכמה, שאילתות, מיגרציות
`expertise-observability`	לוגים, בלי מידע אישי, OpenTelemetry	כל פעולה ב-apps/api
`expertise-react-nextjs`	Server/Client, RTK Query, Mantine RTL, טפסים	קוד ב-apps/web
`expertise-testing`	TDD, Jest + RTL, db-mock, סדר עדיפויות	כל קובץ בדיקה

הסקילים אינם חופפים: `expertise-code-quality` קובע את העקרונות הכלליים, וכל סקיל אחר מוסיף עליהם שכבה מתחומו. בנוסף, שני תרשימים נשמרים ב-`docs/diagrams/` ומתעדכנים בתוך הסקילים: ה-ERD (`expertise-postgres-drizzle`) ותרשים המחלקות (`expertise-api-rest`).